

Automated and Human Proofs in General Mathematics: An Initial Comparison

Jesse Alama¹, Daniel Kühlwein², and Josef Urban^{2,*}

¹ New University of Lisbon

² Radboud University Nijmegen

Abstract. First-order translations of large mathematical repositories allow discovery of new proofs by automated reasoning systems. Large amounts of available mathematical knowledge can be re-used by combined AI/ATP systems, possibly in unexpected ways. But automated systems can be also more easily misled by irrelevant knowledge in this setting, and finding deeper proofs is typically more difficult. Both large-theory AI/ATP methods, and translation and data-mining techniques of large formal corpora, have significantly developed recently, providing enough data for an initial comparison of the proofs written by mathematicians and the proofs found automatically. This paper describes such an initial experiment and comparison conducted over the 50000 mathematical theorems from the Mizar Mathematical Library.

1 Introduction: Automated Theorem Proving in Mathematics

Computers are becoming an indispensable part of many areas of mathematics [6]. As their capabilities develop, human mathematicians are faced with the task of steering, comprehending, and evaluating the ideas produced by computers, similar to the players of chess in recent decades. A notable milestone is the automatically found proof of the Robbins conjecture by EQP [8] and its postprocessing into a human-comprehensible proof by ILF [3] and Mathematica [5]. Especially in small equational algebraic theories (e.g., quasigroup theory), a number of nontrivial proofs have been already found automatically [11], and their evaluation, understanding, and automated post-processing is an open problem [18].

This motivates our interest in making ATP useful also for general mathematics, as done by trained mainstream mathematicians, using standard set-theoretical foundations and a large body of general mathematical knowledge. In the recent years, large general mathematical corpora like the Mizar Mathematical Library (MML) and the Isabelle/HOL library are being made available to automated reasoning and AI methods [14,10], leading to the development of automated reasoning techniques working in large theories with many previous theorems, definitions, and proofs that can be re-used [16,7,9,17]. A recent evaluation (and tuning) of ATP systems on the MML [15]

* J. Alama was funded by the ESF research project *Dialogical Foundations of Semantics*, FCT LogICCC/0001/2007. D. Kühlwein and J. Urban were funded by the NWO projects *Learning2Reason* and *MathWiki*. We thank P. Rudnicki for providing computing support for the work described here, and the anonymous LPAR referees for valuable comments.

has shown that the Vampire/SInE system (winner of all CASC LTB competitions) can already re-prove 39% of the MML’s 50000 theorems when the necessary premises are precisely selected from the human¹ proofs, and about 14% of the theorems when the ATP is allowed to use the whole available library, leading on average to 40000 premises in such ATP problems. In [1] it is further shown on a subset of 2078 MML problems that re-using (generalizing and learning) the knowledge accumulated in previous proofs can further significantly (currently 30–40%, depending on the learning method) improve the performance of combined AI/ATP systems in large-theory mathematics.

This performance, and the recently developed exact proof analysis for the MML [2], allowed an experiment with finding automatically all proofs in the MML by a combination of learning and ATP methods. This is described in Section 2. The 9141 ATP proofs found automatically were then compared using several metrics to the human proofs. This is described in Section 3 and in Section 4.

2 Finding Proofs in the MML with AI/ATP Support

To create a sufficient body of ATP proofs from the MML, we have conducted a large AI/ATP experiment that makes use of several recently developed techniques and significant computational resources. The basic idea of the experiment is to lift to the whole MML (approximately 50000 theorems and more than 100000 premises) the setting used in [1] for large-theory automated proving of 2078 related MML problems. The setting consists of the following three consecutive steps:

- mining the exact minimal proof dependencies from all human-written MML proofs;
- learning premise selection from the minimal proof dependencies;
- using an ATP (Vampire) to prove new conjectures from the best selected premises.

2.1 Mining the Minimal Dependencies from All Human-Written MML Proofs

This process is in detail described in [2], where it is conducted for the 100 initial articles from the MML. The difficulty (and interestingness) consists in (partially brute-force) minimization of the many proof premises (like typing information) that advanced ITPs like Mizar use implicitly, in order to save their users most of the obvious and tedious steps. Conducting this data-mining requires nontrivial transformation of all MML items (theorems, definitions, etc.) into separate micro-articles, followed by the minimization of the micro-articles’ dependencies. This takes several days for all of MML, however the precise information thus obtained gives rise to an unparalleled corpus of data about human-written proofs in the largest available formal body of mathematics. One advantage of this minimization approach over just collecting dependencies from the standard

¹ Mizar proofs are initially human-written, but they are formal and machine-understandable. That allows their automated machine processing and refactoring, which can make them “less human”. Yet, we believe that their classification as “human” is mostly correct for our purposes, and that MML/MPTP is probably the most suitable resource today for attempting this initial comparison of ATP and human proofs.

ITP proof objects is that the latter often include many products of “wasteful” (non-minimized) ITP techniques, like congruence closure over all available ground equalities, and exhaustive application of typing hierarchies. In the final account, the days of computation devoted to minimization pay off, by providing more precise advice for proving new conjectures over the whole MML. An approximate estimate of the computational resources taken by this job is about ten days of full (parallel) CPU load (12 hyperthreading Xeon 2.67 GHz cores, 24 GB RAM) of the Mizar server at the University of Alberta. The resulting minimized dependencies for all MML items can be viewed online at a web page² presenting them.

2.2 Learning Premise Selection from Proof Dependencies

To learn premise selection from proof dependencies, one characterizes all MML formulas by suitable features, and feeds them (together with the detailed proof information) to a machine learning system that is trained to advise premises for later conjectures. Formula symbols have been used previously for this task in [14]. Thanks to sufficient hardware being available, we have for the first time included also term features generated by the MaLARea framework, added to it in 2008 [16] for experiments with smaller subsets of MML. Thus, for each MML formula we include as its characterization also all the subterms and subformulas contained in the formula, which makes the learning and prediction more precise. To our surprise, the EPROVER-based [13] utility that consistently numbers all shared terms in all formulas, written for this purpose in 2008 by the last author, scaled without problems to the whole MML, and this feature-generation phase took only minutes. This created over one million learning features that are used to characterize all the mathematics in MML for learning. We have also briefly explored using validity in finite models (again, introduced in MaLARea in 2008, building on Pudlák’s previous work [12]) as a more semantic way of characterizing formulas. However, this has turned out to be very time-consuming, most likely caused by the LADR-based `clausefilter` utility struggling with evaluating in the models some complicated (containing many quantifiers) mathematical formulas from the MML. Clearly, further optimizations are needed for extracting such semantic characterizations for all of MML. Even without such features, the machine learning was already pushed to the limit. The more advanced kernel-based multi-output ranker developed in [1] turned out to be too slow and memory-exhaustive to handle over one million features and over hundred thousand training examples without further improvements. The SNoW system used in naive Bayes mode took several gigabytes of RAM to train on the data, and on average about a second (ca a day of computation for all of MML) to produce a premise prediction for each MML problem (based always on incremental training³ on all previous MML proofs characterized by their exact dependencies, and the formula features). It seems that pushing such AI methods to handle, for example, the mathematics from the whole arXiv.org, Elsevier, or Springer corpora (if ever (semi-)formalized), will require some nontrivial scaling up, going for technologies developed by Google and similar

² <http://mizar.cs.ualberta.ca/mizar-items>

³ In the incremental learning mode, the evaluation and training are done at the same time for each example, hence there was no extra time taken by training.

companies for dealing with very large learning tasks. The results of this run are SNoW premise predictions for all of MML, available online as the raw SNoW output at our web page⁴, and also postprocessed into corresponding ATP problems (see below).

2.3 Using ATPs to Prove the Conjectures from the Selected Premises

As the MML grows from axioms of set theory to advanced mathematics, it gives rise to a chronological ordering of its theorems. When a new theorem C is conjectured, all the previous theorems and definitions are available as premises, and all the previous proofs are used to learn which of these premises are relevant for C . The SNoW system provides a ranking of all premises, and the best premises are given to an ATP which attempts a proof of C .

There are many ways how to organize several ATP systems to attack C , and how to try with different numbers of the best-ranked premises and with different time limits. Several such policies are implemented in MaLAREa, which can run for months on the whole MML while still producing new proofs. Because our main interest here was to get a sufficient body of automatically found proofs for their further evaluation, we have fixed the ATP system to be Vampire (version 1.8), and we have always used 200 best premises and a time limit of 20 seconds. A 12-core 2.67 GHz Xeon server at University of Alberta was used for (parallelized) proving, which took about a day in real time. This has produced 9141 automatically found proofs that we further analyze. The overall success rate is over 18% of theorems proved which is so far the best result on the whole MML, but we have not really focused yet on getting this as high as possible. For example, running Vampire in parallel with both 40 and 200 best recommended premises has been shown to significantly improve the success rate, and a preliminary experiment with the Z3 solver has provided another two thousand proofs from the problems with 200 best premises. Unfortunately, Z3 does not (yet) print the names of premises used in the proofs, so its proofs would not be directly usable for the analysis that is conducted here. When using a large number of premises, an ATP proof can often contain unnecessary premises. To get more closely the set of premises that an ATP actually needs, we always re-run the ATP only with the premises that were used in the first run. For Vampire, such minimization is quite significant. The resulting ATP problems are also available online⁵ for further experiments, as well as all the 9141 proofs found.⁶

3 Proof Metrics

We thus have, for 9141 Mizar theorems ϕ , the set of premises that were used in the (minimized) ATP proof of ϕ . Each ATP proof was found completely independently of its Mizar proof, i.e., no information (e.g., about the premises used) from the Mizar proof was transferred to the ATP proof.⁷ This gives us a notion of dependency for

⁴ http://mizar.cs.ualberta.ca/~mptp/proofcomp/snow_predictions.tar.gz

⁵ <http://mizar.cs.ualberta.ca/~mptp/proofcomp/advised200f1.tar.gz>

⁶ <http://mizar.cs.ualberta.ca/~mptp/proofcomp/proved200f1min.tar.gz>

⁷ The ATP proofs are however always based on the same state of previous theory and proof knowledge. This could be further relaxed in future experiments.

Mizar theorems, derived from an ATP. From the Mizar proof dependency analysis we also know precisely what Mizar items are needed for a given Mizar (human-written) proof to be successful.

Definition 1. For a Mizar theorem ϕ , let $P_H(\phi)$ be the minimal set of premises needed for the success of the human proof of ϕ . Let $P_A(\phi)$ be the set of premises used by an ATP to prove ϕ .

This gives rise to the notions of “immediate dependence” and “indirect dependence” of one Mizar item a upon another Mizar item b :

Definition 2. For Mizar items a and b , $a <_1 b$ means that a immediately depends on b ($b \in P_H(a)$). Let $<$ be the transitive closure of $<_1$, $\leq$ its reflexive version, and let $P_H^*(a) := \{b : b < a\}$. For a set S of items, let $P_H^*(S) := \{b : \exists a \in S : b \leq a\}$.

While theoretically, there are multiple versions of $<_1$ and $<$ induced by different (ATP, Mizar) proofs, unless we explicitly state otherwise, these relations will always refer to the dependencies derived from the Mizar proofs. The pragmatic reason is that we do not have an ATP proof for all Mizar items,⁸ and hence we do not have the full dependency graph induced by ATP proofs. Also, the way ATP proofs were produced was by always relying on the previous Mizar theorems and dependency data, therefore it makes sense to also use the Mizar data for the transitive closure.

We now define two metrics (and their transitive, human, and ATP versions) D (Dependencies) and L (Length) measuring the complexity of proofs of Mizar theorems. The human alternative of D just (recursively) counts all proof dependencies, based on the full Mizar dependency graph. The ATP alternative uses the ATP proof, and its transitive version recurses using the (full) Mizar dependency graph.

Definition 3. For each Mizar item a , let $D_H(a) := |P_H(a)|$ and $D_H^*(a) := |P_H^*(a)|$. For a set S of items, let $D_H^*(S) := |P_H^*(S)|$. For each ATP-proved theorem a , let $D_A(a) := |P_A(a)|$, and $D_A^*(a) := D_H^*(P_A(a))$.

The second metric L adds weighting by (recursive) proof complexity, which is for the Mizar proofs computed using the assumption that the Mizar weak refutational checker enforces a relatively uniform degree of derivational complexity on all Mizar proof steps, which roughly correspond to proof lines in Mizar formalizations. For the ATP version, we make a similar assumption that the complexity of ATP proof steps is roughly uniform.⁹ For the comparison with human proofs, we first need to define a conversion ratio $c_{A/H}$ between the number of ATP inference lines and the corresponding number of Mizar proof lines. This is pragmatically estimated as the average of such ratios for all the proofs where the ATP used the same premises as the Mizar proof. The actual value computed (based on 1223 proofs where $P_A(a) = P_H(a)$) is $c_{A/H} = 81.99$. Formally:

⁸ We have limited the ATP experiment to Mizar *theorems*, so even with perfect ATP success rate we would still miss for example all ATP dependencies of Mizar definitions, that often require proofs of existence, uniqueness, etc.

⁹ The precision of such metrics could be further improved, for example by expanding Vampire proofs into the (really uniform) detailed proof objects developed for Otter/Prover9/IVY.

Definition 4. For a Mizar-proved item a , let $L_H(a)$ be the number of Mizar lines of code used to prove a (direct Mizar proof length). For each ATP-proved item a , let $L_A^0(a)$ be the number of steps in the ATP proof. Let $E_{H=A} := \{a: P_A(a) = P_H(a)\}$ (items whose ATP and Mizar proofs use the same premises). Let $c_{A/H} := 1/|E_{H=A}| * \sum_{a \in E_{H=A}} L_A^0(a)/L_H(a)$ (the length conversion ratio). Finally, we define the normalized ATP proof length as $L_A(a) := L_A^0(a)/c_{A/H}$. For a set of items S , let again $L_H(S) := \sum_{a \in S} L_H(a)$.

Definition 5. For a Mizar theorem a we define $L_H^*(a) := L_H(a) + L_H(P_H^*(a))$. If we have an ATP proof, we define $L_A^*(a) := L_A(a) + L_H(P_H^*(P_A(a)))$.

The reason for using $L_H()$ and $P_H^0()$ in the recursive part of L_A^* is again the fact that we only have the complete line count information for the Mizar proofs. Note that both in D^* and in L^* we always count any lemma on the transitive proof path exactly once. We believe that this approach captures the mathematician’s intuition of *proof complexity* as the *set* of “the proofs that need to be understood” rather than as their multiset. This could be further explored by various cognitive experiments.

4 Evaluation

The metrics developed above were used for an initial comparison of the Mizar and ATP proofs. The detailed evaluation data corresponding to this section are available online.¹⁰ First we analyze the data based on the relation between $P_H()$ and $P_A()$. For each Mizar theorem ϕ that can be proved by an ATP, we have either $P_H(\phi) = P_A(\phi)$, $P_H(\phi) \subset P_A(\phi)$, $P_A(\phi) \subset P_H(\phi)$, or neither set is included in the other. Let us say that two sets A and B are *orthogonal* if neither $A \subseteq B$, nor $B \subseteq A$. The statistics is given in Table 1.

Table 1. Premise statistics for the categories

Category	Cases	Max A	Min A	Avg A	Max H	Min H	Avg H	Max $ H - A $	Avg $ H - A $
$P_H(a) = P_A(a)$	1223	7	0	2.18	7	1	2.11	0	0
$P_A(a) \subset P_H(a)$	1980	12	0	2.20	59	1	5.58	58	3.40
$P_H(a) \subset P_A(a)$	386	89	1	6.24	10	1	2.41	83	3.88
Orthogonal	5552	63	1	5.22	58	1	6.33	60	3.86

While the *orthogonal* category is largest as was expected, it is surprising to see more than 10% of the proofs to be the same according to the D metric. It is even more surprising to see that 1980 ATP proofs (21.66%) are shorter according to the D metric. An initial analysis suggested (at least) the following explanations:

- The ATP is naturally oriented towards as short proofs as possible. Getting involved proofs with many premises is hard, and it may well be the main reason of ATP failure outside the 9141 proved theorems.

¹⁰ http://mizar.cs.ualberta.ca/~mptp/proofcomp/metrics_evaluation.xls

- In many cases, a human formalizer can overlook the fact that the same or very similar theorem is already in the library.¹¹ An example is the theorem LOPBAN_3:24¹² which required a 20-line proof in Mizar, but the ATP found an earlier more general theorem BHSP_4:3 that (using additional typing information) provides an almost immediate proof.
- ATPs work in untyped first-order logic, and they are not constrained by the Mizar's (and other ITPs') requirement that all types should be inhabited. For example, Mizar proof checking of GOEDELCP:1¹³ fails if two type non-emptiness declarations are removed, because the formula is no longer well-typed. The ATP proof however does not need any of them.

An interesting case is when the ATP finds an inventive way how to re-use previous lemmas. Sometimes enough knowledge about advanced concepts is already developed that can be used for their quite simple (“algebraic”) manipulation, abstracting from their definitions. An example is COMSEQ_3:40¹⁴, proving the relation between the limit of a complex sequence and its real and imaginary parts. The human proof expands the definitions (finding a suitable n for a given ϵ). The ATP just notices that this kind of groundwork was already done in a “similar” case COMSEQ_3:39¹⁵, and notices the “similarity” (algebraic simplification) provided by COMPLEX1:28¹⁶. Such manipulations can be used (if noticed!) to avoid the “hard thinking” about the epsilons in the definitions.

4.1 Comparing Weights

For a Mizar theorem ϕ , a large difference between $L_H^*(\phi)$ and $L_A^*(\phi)$ is a sign that the ATP of ϕ is importantly different from the human Mizar proof of ϕ . The Table 2 shows that with exception of the $P_H(a) = P_A(a)$ case, which we used to define $c_{A/H}$, the ATP proofs have on average higher recursive complexity L^* than the corresponding human proofs. Again, we have found several explanations:

- Some cases are due to the failure in minimization of the ATP proofs. For example, the ATP proof of FUNCT_7:20¹⁷ reports 40 premises and 178715 ATP (non-normalized) proof steps, largely coming from recent addition of BDDs to Vampire.

¹¹ From this point of view, this analysis is conducted at the right time, because the ATP service is starting to be used by authors, and such simple repetitions will be prevented by it.

¹² http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t24.lopban_3 The theorem says that the partial-sums operator on normed space sequences commutes with multiplication by a scalar.

¹³ <http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t1.goedelcp>

¹⁴ http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t40.comseq_3

¹⁵ http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t39.comseq_3

¹⁶ <http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t28.complex1>

¹⁷ http://mizar.cs.ualberta.ca/~MPTP/cgi-bin/browserefs.cgi?refs=t20.funct_7

Table 2. Recursive line count/proof step statistics (L^*) for the categories

Category	Cases	Max A	Min A	Avg A	Max H	Min H	Avg H	Max $ H - A $	Avg $ H - A $
$P_H(a) = P_A(a)$	1223	140176	7	7390.77	140438	1	7385.06	6210	0
$P_A(a) \subset P_H(a)$	1980	40653	26	7373.31	32652	1	6167.73	40626	1220.52
$P_H(a) \subset P_A(a)$	386	162308	9	14155.3	162532	3	14768.3	35536	632.329
Orthogonal	5552	139935	13	9893.04	140172	3	9828.81	75114	910.744

- Most of the cases again seem to be due to the ATPs tendency to get a short proof by advanced lemmas, rather than getting into longer proofs by expanding the definitions. The lemmas typically recursively use the basic definitions anyway, and their line complexity is then a net contribution to the ATP proof’s recursive complexity.

5 Conclusion

While ATPs in general large-theory formal mathematics are becoming clearly useful, an initial proof analysis using quite straightforward metrics has not yet found any highly surprising ATP proofs. Clearly, the general large-theory mathematical setting is still quite far from producing automated proofs of the order of complexity that some specialized algebraic theories enjoy. On the other hand, the ATPs have found a surprising number of proofs that are shorter than the mathematicians’ version, and the ATP’s ideas are very often relevant. Unlike humans, the combined AI/ATP stack learns new lemmas and new proofs immediately, and this results in their more extensive use and significantly higher value of L^* . An ATP working in unsorted FOL can sometimes find proofs that, in some sense, get to the “mathematical heart” of a theorem without first going through the syntactic hoops of ensuring that terms have suitable sorts. The tools produced for our experiments can produce information that is useful for maintainers of large formal libraries. We found cases where an ATP was able to find a significantly shorter proof—sometimes employing only one premise—compared to a human proof. At times, such highly efficient ATP proofs were due to duplication in the library or failure to use a generalization to prove a special case. Finally, our work comparing different proofs could provide a practical “test bed” for theoretical criteria of proof identity [4].

References

1. Alama, J., Kühlwein, D., Tsvitsivadze, E., Urban, J., Heskes, T.: Premise selection for mathematics by corpus analysis and kernel methods. CoRR, abs/1108.3446
2. Alama, J., Mamane, L., Urban, J.: Dependencies in formal mathematics. CoRR, abs/1109.3687 (2011), <http://arxiv.org/abs/1109.3687>
3. Dahn, I.: Robbins algebras are Boolean: A revision of McCune’s computer-generated solution of Robbins problem. Journal of Algebra 208, 526–532 (1998)
4. Dosen, K.: Identity of proofs based on normalization and generality. Bulletin of Symbolic Logic 9, 477–503 (2003)
5. Fitelson, B.: Using *Mathematica* to understand the computer proof of the Robbins Conjecture. Mathematica In Education and Research 7(1) (1998)

6. Hales, T.: Mathematics in the age of the Turing Machine. Lecture Notes in Logic in Commemoration of the Centennial of the Birth of Alan Turing (to appear, 2012)
7. Hoder, K., Voronkov, A.: Sine Qua Non for Large Theory Reasoning. In: Bjørner, N., Sofronie-Stokkermans, V. (eds.) CADE 2011. LNCS, vol. 6803, pp. 299–314. Springer, Heidelberg (2011)
8. McCune, W.W.: Solution of the Robbins Problem. *Journal of Automated Reasoning* 19(3), 263–276 (1997)
9. Meng, J., Paulson, L.C.: Lightweight relevance filtering for machine-generated resolution problems. *J. Applied Logic* 7(1), 41–57 (2009)
10. Paulson, L.C., Blanchette, J.: Three years of experience with Sledgehammer, a practical link between automated and interactive theorem provers. In: 8th IWIL (2010)
11. Phillips, J.D., Stanovský, D.: Automated Theorem Proving in Loop Theory. In: Sutcliffe, G., Colton, S., Schulz, S. (eds.) ESARM. CEUR Workshop Proceedings, vol. 378, pp. 42–53. CEUR-WS.org (2008)
12. Pudlák, P.: Semantic selection of premisses for automated theorem proving. In: Sutcliffe, G., Urban, J., Schulz, S. (eds.) ESARLT. CEUR Workshop Proceedings, vol. 257. CEUR-WS.org (2007)
13. Schulz, S.: E – a brainiac theorem prover. *J. of AI Communications* 15(2-3), 111–126 (2002)
14. Urban, J.: MPTP 0.2: Design, implementation, and initial experiments. *J. Autom. Reasoning* 37(1-2), 21–43 (2006)
15. Urban, J., Hoder, K., Voronkov, A.: Evaluation of Automated Theorem Proving on the Mizar Mathematical Library. In: Fukuda, K., van der Hoeven, J., Joswig, M., Takayama, N. (eds.) ICMS 2010. LNCS, vol. 6327, pp. 155–166. Springer, Heidelberg (2010)
16. Urban, J., Sutcliffe, G., Pudlák, P., Vyskočil, J.: MaLAREa SG1–Machine learner for automated reasoning with semantic guidance. In: IJCAR, pp. 441–456 (2008)
17. Urban, J., Vyskočil, J., Štěpánek, P.: MaLeCoP Machine Learning Connection Prover. In: Brunnler, K., Metcalfe, G. (eds.) TABLEAUX 2011. LNCS, vol. 6793, pp. 263–277. Springer, Heidelberg (2011)
18. Vyskočil, J., Stanovský, D., Urban, J.: Automated Proof Compression by Invention of New Definitions. In: Clarke, E.M., Voronkov, A. (eds.) LPAR-16 2010. LNCS, vol. 6355, pp. 447–462. Springer, Heidelberg (2010)